VIBE CODING LAB – FROGGER GAME

Chat History and Implementation Report

Project: Python Pygame Frogger Game
Lab: Lab 4 – Vibe Coding
Objective: Identify and fix bugs in an existing game, implement the required features using AI-assisted coding, test the changes, and push the updated code to a personal GitHub repository.

Name: Shantheri Shenoy

SRN :PES1UG24CS428

Section :H

1. ORIGINAL LAB REQUIREMENTS

The lab handout specifies the following workflow:

1. Open the assigned repository.
2. Read the README file to understand the requirements.
3. Clone or fork the repository.
4. Run the original Python game.
5. Record a 10-second video before making any changes.
6. Use Vibe Coding tools to identify and fix bugs.
7. Implement the features listed in the README one by one.
8. Record another 10-second video after completing the changes.
9. Push the updated code to a personal GitHub repository.
10. Do not raise a pull request to the original SETAPESU26 repository.

Required deliverables

- Video of the original game.
- Video of the completed game.
- Updated source code.
- AI chat history exported as a DOC or PDF.

2. PROJECT SETUP

User

I want to work on the Frogger thing. Tell me properly what I am supposed to do, step by step.

AI Assistant

The implementation should be completed in the following order:

1. Extract the provided ZIP file.
2. Open the project folder in Visual Studio Code.
3. Install the required Python dependencies.
4. Run the original game.
5. Record the original game before changing the code.
6. Fix the vehicle collision detection.
7. Add three lives and frog respawning.
8. Add goal detection and score tracking.
9. Add a 30-second countdown timer.
10. Test the completed game.
11. Record the final video.
12. Export the AI conversation and push the code to GitHub.

Setup commands

Check the Python installation:

py --version

Install the project dependencies:

py -m pip install -r requirements.txt

Run the original game from the frogger directory:

py main.py

The original game must be recorded before any code modifications.

Original video filename: frogger_before.mp4

3. TASK 1 – FIX VEHICLE COLLISION DETECTION

3.1 Problem identification

User prompt

I am working on a Python Pygame Frogger game for a Vibe Coding lab.

Task 1: Fix vehicle collision detection.

Currently, the collision code calculates the vehicle's grid column using int(v.x // CELL_SIZE) and compares it with the frog's column and row. This can miss collisions because vehicles move continuously in pixel coordinates and have different widths.

Modify only the necessary code so that collision detection uses accurate rectangle intersection between the frog and every vehicle. Use frog.get_rect(CELL_SIZE) and vehicle.get_rect(CELL_SIZE), and return True if any rectangles overlap.

Preserve the existing project structure and function signature. Explain the change and provide the complete corrected collisions.py file.

3.2 AI response – Explanation

The original collision detection compared grid positions rather than the actual rectangular areas occupied by the frog and vehicles.

This could miss a collision when a vehicle overlapped the frog but its calculated grid column differed from the frog's column.

The proposed solution uses Pygame's rectangle-intersection method, colliderect().

3.3 Corrected collisions.py

```
"""

collisions: frog-vs-vehicle collision detection.

"""


CELL_SIZE = 50



def check_collision(frog, vehicles):
    """
    Returns True if the frog overlaps any vehicle.
    Uses rectangle intersection for accurate collision detection.
```

```
    """
    frog_rect = frog.get_rect(CELL_SIZE)

    for v in vehicles:
        vehicle_rect = v.get_rect(CELL_SIZE)

        if frog_rect.colliderect(vehicle_rect):
            return True

    return False
```

3.4 Explanation of the corrected code

Step 1: Obtain the frog's rectangle.

frog_rect = frog.get_rect(CELL_SIZE)

This obtains the frog's rectangular boundary.

Step 2: Examine every vehicle.

for v in vehicles:

The function checks the frog against each vehicle in the game.

Step 3: Obtain the vehicle's rectangle.

vehicle_rect = v.get_rect(CELL_SIZE)

This obtains the vehicle's rectangular boundary, including its width.

Step 4: Check for an intersection.

if frog_rect.colliderect(vehicle_rect):
    return True

If the rectangles overlap, the function returns True.

If no vehicle overlaps the frog, the function returns False.

3.5 User question – What exactly changed?

User

I am not getting what changed now, like which overlap is detected?

AI Assistant

The code checks whether the frog's rectangle intersects a vehicle's rectangle.

If the rectangles are separate, the function returns False.

If the rectangles overlap, the function returns True.

The collision check is based on the actual rectangular areas of the objects, rather than only their calculated grid positions.

3.6 User question – Does it need to intersect in the same line?

User

In the same line it should intersect or in any of the line?

AI Assistant

The rectangles can intersect at any position. The code does not require the frog and vehicle to have identical grid rows or columns.

The important condition is that their actual rectangular areas overlap.

If the rectangles merely touch at their edges without overlapping, Pygame's colliderect() returns False.

3.7 User question – Is the collision detected in the recording?

User

See, is that detected here?

AI Assistant

The recording showed the frog and vehicles moving, but it did not clearly establish that their rectangles overlapped at the exact moment of collision.

To verify the fix, move the frog directly into a vehicle and check whether the existing collision response occurs.

3.8 User observation – Collision already happened before

User

When it hits any rectangle, collision was happening even before and now also.

AI Assistant

If both versions detect the collision you tested, the particular test may not expose the original bug.

The intended change is:

- **Old method:** Compare calculated grid positions.
- **New method:** Check actual rectangle intersection.

A useful test would demonstrate a case in which the old implementation misses a collision and the new implementation detects it.

The videos alone do not prove that the bug was fixed.

3.9 Additional debugging

User

I think when there is a block just above that, it was not becoming out at that time.

AI Assistant

To investigate, temporary debugging statements can be added inside the vehicle loop:

print("Frog:", frog_rect, "Vehicle:", vehicle_rect)

print("Overlap:", frog_rect.colliderect(vehicle_rect))

These statements display the frog's rectangle, the vehicle's rectangle, and the overlap result.

If the rectangles overlap but the frog does not reset, the problem may be in the game engine's collision handling rather than in collisions.py.

If the result is False when the objects appear to overlap visually, the rectangle definitions should be inspected.

4. TASK 2 – ADD THREE LIVES AND FROG RESPAWNING

4.1 User prompt

I am working on a Python Pygame Frogger game for my Vibe Coding lab.

Task 2: Add a lives system and frog respawn.

Requirements:

1. The player starts with 3 lives.
2. Every vehicle collision reduces lives by exactly one.
3. After a collision, the frog respawns at its original starting position.
4. Prevent one collision from deducting multiple lives across consecutive frames. Use a suitable collision cooldown or temporary invulnerability mechanism.
5. When lives reach zero, display a Game Over state and stop normal gameplay.

6. Pressing R restarts the game and restores all 3 lives.
7. Display the remaining lives on the game window.

Preserve my existing project structure and the collision detection fix already made in collisions.py. Do not implement the score or timer yet.

First inspect my existing game_engine.py, frog.py, and renderer.py. Explain which files need changes and why. Then provide the complete updated code for each changed file. Do not invent classes, methods, or attributes that do not exist in my project.

4.2 AI response – Files involved

The proposed changes were:

File	Purpose
game_engine.py	Manage lives, collisions, respawning, Game Over, and restart.
renderer.py	Display lives and the Game Over message if existing rendering functions are insufficient.
frog.py	Reuse the existing reset() method to return the frog to its starting position.
collisions.py	Preserve the rectangle-based collision fix.

The implementation reused the existing draw_text() and draw_banner() functions from the renderer.

4.3 Updated game_engine.py – Task 2

"""

GameEngine: owns the frog and all vehicles, and runs the game logic.

Includes lives, collision respawning, Game Over, and restart.

"""

```
import random
import pygame

from game.frog import Frog
from game.vehicle import Vehicle
from game.collisions import check_collision
```

```
from game.renderer import (
    GRID_COLS, GRID_ROWS, GOAL_ROW, ROAD_ROWS, START_ROW,
    CELL_SIZE, WIDTH, HEIGHT,
)

LANE_SPEEDS = [1.5, -2, 2, -2.5, 1.5, -2]



class GameEngine:
    def __init__(self):
        self.lives = 3
        self.game_over = False
        self.invulnerable_until = 0
        self._build_entities()

    def _build_entities(self):
        start_col = GRID_COLS // 2

        self.frog = Frog(
            col=start_col,
            row=START_ROW,
            start_col=start_col,
            start_row=START_ROW,
            cols=GRID_COLS,
            start_row_limit=START_ROW,
        )

        frog_x_range = (
```

```
    start_col * CELL_SIZE,
    start_col * CELL_SIZE + CELL_SIZE,
)

self.vehicles = []

for i, row in enumerate(ROAD_ROWS):
    speed = LANE_SPEEDS[i % len(LANE_SPEEDS)]
    vehicle_width = 40 if i % 2 == 0 else 70
    spacing = 300
    count = 2

    for _attempt in range(20):
        phase = random.randint(0, spacing - 1)
        positions = []
        safe = True

        for n in range(count):
            offset = phase + n * spacing
            x = (
                offset
                if speed > 0
                else WIDTH - offset - vehicle_width
            )
            positions.append(x)

            if not (
                x + vehicle_width <= frog_x_range[0]
```

```
                or x >= frog_x_range[1]
            ):
                safe = False

        if safe:
            break

    for x in positions:
        self.vehicles.append(
            Vehicle(
                x=x,
                row=row,
                width=vehicle_width,
                height=CELL_SIZE - 8,
                speed=speed,
            )
        )

def handle_keydown(self, key):
    if key == pygame.K_r:
        self.lives = 3
        self.game_over = False
        self.invulnerable_until = 0
        self._build_entities()
        return

    if self.game_over:
        return
```

```
    if key == pygame.K_UP:
        self.frog.move(0, -1)
    elif key == pygame.K_DOWN:
        self.frog.move(0, 1)
    elif key == pygame.K_LEFT:
        self.frog.move(-1, 0)
    elif key == pygame.K_RIGHT:
        self.frog.move(1, 0)

def update(self):
    if self.game_over:
        return

    for v in self.vehicles:
        v.update(road_width_px=WIDTH)

    current_time = pygame.time.get_ticks()

    if (
        current_time >= self.invulnerable_until
        and check_collision(self.frog, self.vehicles)
    ):
        self.lives -= 1
        self.frog.reset()
        self.invulnerable_until = current_time + 1000

        if self.lives <= 0:
```

```
            self.lives = 0
            self.game_over = True

        return

    if self.frog.row == GOAL_ROW:
        self.frog.reset()

def draw(self, surface, font):
    from game import renderer

    renderer.draw_scene(surface, self.frog, self.vehicles)

    renderer.draw_text(
        surface,
        font,
        f"Lives: {self.lives}",
        (10, 10),
    )

    renderer.draw_text(
        surface,
        font,
        "Arrow keys to move. R to restart.",
        (10, HEIGHT - 24),
    )

    if self.game_over:
```

```
        renderer.draw_banner(
            surface, font, "GAME OVER - Press R to restart"
        )
```

4.4 Explanation

Lives initialization

self.lives = 3

The player begins with three lives.

Collision handling

```
if (
    current_time >= self.invulnerable_until
    and check_collision(self.frog, self.vehicles)
):
```

The collision is processed only when the player is not temporarily invulnerable.

Deduct one life

self.lives -= 1

One life is deducted for a processed collision.

Respawn

self.frog.reset()

The frog returns to its original starting position.

Collision cooldown

self.invulnerable_until = current_time + 1000

This provides one second of temporary invulnerability after a collision.

Game Over

```
if self.lives <= 0:
    self.lives = 0
    self.game_over = True
```

When lives reach zero, the game enters the Game Over state.

Restart

Pressing R restores three lives and rebuilds the game entities.

4.5 Task 2 testing checklist

The following tests were identified:

1. Game starts with 3 lives.
2. First collision reduces lives to 2.
3. Frog respawns at its starting position.
4. A single collision does not repeatedly deduct lives immediately.
5. Second collision reduces lives to 1.
6. Third collision displays Game Over.
7. Pressing R restarts the game with 3 lives.

The task should be considered complete only after testing these behaviors.

5. TASK 3 – ADD GOAL DETECTION AND SCORE TRACKING

5.1 User request

The updated Frogger game runs. Help me implement Task 3: goal detection and score tracking, preserving the existing lives system and collision fix.

5.2 AI response – Implementation plan

The following features were added:

1. A score initialized to zero.
2. Goal detection using GOAL_ROW.
3. Ten points awarded for each successful goal.
4. A temporary goal-completed message.
5. Frog respawning after a successful goal.
6. Score reset when R is pressed.
7. Preservation of the lives and collision handling.

5.3 Updated game_engine.py – Task 3

"""

GameEngine: owns the frog and all vehicles.

```
Includes lives, collision respawning, goal detection, and score tracking.
"""

import random
import pygame

from game.frog import Frog
from game.vehicle import Vehicle
from game.collisions import check_collision
from game.renderer import (
    GRID_COLS, GRID_ROWS, GOAL_ROW, ROAD_ROWS, START_ROW,
    CELL_SIZE, WIDTH, HEIGHT,
)

LANE_SPEEDS = [1.5, -2, 2, -2.5, 1.5, -2]
GOAL_POINTS = 10
GOAL_MESSAGE_DURATION = 1500


class GameEngine:
    def __init__(self):
        self.lives = 3
        self.score = 0
        self.game_over = False
        self.invulnerable_until = 0
        self.goal_message_until = 0
        self._build_entities()
```

```
def _build_entities(self):
    start_col = GRID_COLS // 2

    self.frog = Frog(
        col=start_col,
        row=START_ROW,
        start_col=start_col,
        start_row=START_ROW,
        cols=GRID_COLS,
        start_row_limit=START_ROW,
    )

    frog_x_range = (
        start_col * CELL_SIZE,
        start_col * CELL_SIZE + CELL_SIZE,
    )

    self.vehicles = []

    for i, row in enumerate(ROAD_ROWS):
        speed = LANE_SPEEDS[i % len(LANE_SPEEDS)]
        vehicle_width = 40 if i % 2 == 0 else 70
        spacing = 300
        count = 2

        for _attempt in range(20):
            phase = random.randint(0, spacing - 1)
            positions = []
```

```
            safe = True

            for n in range(count):
                offset = phase + n * spacing
                x = (
                    offset
                    if speed > 0
                    else WIDTH - offset - vehicle_width
                )
                positions.append(x)

                if not (
                    x + vehicle_width <= frog_x_range[0]
                    or x >= frog_x_range[1]
                ):
                    safe = False

            if safe:
                break

        for x in positions:
            self.vehicles.append(
                Vehicle(
                    x=x,
                    row=row,
                    width=vehicle_width,
                    height=CELL_SIZE - 8,
                    speed=speed,
```

```
            )
        )

def handle_keydown(self, key):
    if key == pygame.K_r:
        self.lives = 3
        self.score = 0
        self.game_over = False
        self.invulnerable_until = 0
        self.goal_message_until = 0
        self._build_entities()
        return

    if self.game_over:
        return

    if key == pygame.K_UP:
        self.frog.move(0, -1)
    elif key == pygame.K_DOWN:
        self.frog.move(0, 1)
    elif key == pygame.K_LEFT:
        self.frog.move(-1, 0)
    elif key == pygame.K_RIGHT:
        self.frog.move(1, 0)

def update(self):
    if self.game_over:
        return
```

```
    for v in self.vehicles:
        v.update(road_width_px=WIDTH)

    current_time = pygame.time.get_ticks()

    if (
        current_time >= self.invulnerable_until
        and check_collision(self.frog, self.vehicles)
    ):
        self.lives -= 1
        self.frog.reset()
        self.invulnerable_until = current_time + 1000

        if self.lives <= 0:
            self.lives = 0
            self.game_over = True

        return

    if self.frog.row == GOAL_ROW:
        self.score += GOAL_POINTS
        self.goal_message_until = (
            current_time + GOAL_MESSAGE_DURATION
        )
        self.frog.reset()

def draw(self, surface, font):
```

```
    from game import renderer

    renderer.draw_scene(surface, self.frog, self.vehicles)

    renderer.draw_text(
        surface, font, f"Lives: {self.lives}", (10, 10)
    )
    renderer.draw_text(
        surface, font, f"Score: {self.score}", (150, 10)
    )
    renderer.draw_text(
        surface, font,
        "Arrow keys to move. R to restart.",
        (10, HEIGHT - 24),
    )

    if self.game_over:
        renderer.draw_banner(
            surface, font, "GAME OVER - Press R to restart"
        )
    elif pygame.time.get_ticks() < self.goal_message_until:
        renderer.draw_banner(
            surface, font, f"GOAL REACHED! +{GOAL_POINTS} points"
        )
```

5.4 Explanation of the score logic

The score is initialized using:

self.score = 0

The score increases when the frog reaches the goal row:

```
if self.frog.row == GOAL_ROW:
    self.score += GOAL_POINTS
    self.frog.reset()
```

The constant is:

```
GOAL_POINTS = 10
```

Therefore, each successful goal adds ten points.

The game displays the score with:

```
renderer.draw_text(
    surface, font, f"Score: {self.score}", (150, 10)
)
```

The restart logic resets the score:

```
self.score = 0
```

5.5 Task 3 testing checklist

1. Game starts with score 0.
2. Score is visible during gameplay.
3. Reaching the goal adds 10 points.
4. A goal-completed message appears.
5. Frog returns to the starting position.
6. A second successful goal increases the score to 20.
7. Vehicle collisions still deduct lives.
8. Game Over still works.
9. Pressing R resets lives to 3 and score to 0.

6. TASK 4 – ADD A 30-SECOND COUNTDOWN TIMER

6.1 User prompt

I am working on my existing Python Pygame Frogger game. Tasks 1–3 are implemented: rectangle-based vehicle collision detection, a 3-life system with respawn, and goal detection with score tracking.

Task 4: Add a 30-second countdown timer.

Requirements:

1. Each attempt starts with 30 seconds.
2. Display the remaining time on the game window.
3. Calculate elapsed time using pygame.time.get_ticks() or another real-time method, not frame counts.
4. When time reaches zero, end the current attempt, deduct exactly one life, and respawn the frog.
5. Prevent the same timeout from deducting multiple lives.
6. Reset the timer after a collision, timeout, or successful goal.
7. Preserve the existing collision detection, score, lives, Game Over, and restart behavior.
8. Pressing R must reset lives to 3, score to 0, and the timer to 30 seconds.
9. When lives reach zero, display Game Over and stop normal gameplay.

Inspect my existing code before changing it. Modify only the necessary files, preserve the current project structure, and provide the complete updated code for each changed file. Explain how to test every requirement. Do not break the working features.

6.2 AI response – Timer design

The timer uses pygame.time.get_ticks() to measure real elapsed time.

The implementation stores the beginning of each attempt in self.attempt_started_at.

The remaining time is calculated as:

Remaining time = 30 seconds − elapsed time

A timeout calls the same attempt-ending logic as a vehicle collision. This deducts one life, resets the frog, and restarts the timer.

A successful goal adds points and resets the timer without deducting a life.

6.3 Updated game_engine.py – Task 4

"""

GameEngine: owns the frog and all vehicles.

Includes collision detection, lives, score, countdown timer,

respawning, Game Over, and restart.

"""

```
import random
import pygame

from game.frog import Frog
from game.vehicle import Vehicle
from game.collisions import check_collision
from game.renderer import (
    GRID_COLS, GRID_ROWS, GOAL_ROW, ROAD_ROWS, START_ROW,
    CELL_SIZE, WIDTH, HEIGHT,
)

LANE_SPEEDS = [1.5, -2, 2, -2.5, 1.5, -2]

GOAL_POINTS = 10
ATTEMPT_TIME_MS = 30_000
GOAL_MESSAGE_DURATION = 1500
COLLISION_COOLDOWN_MS = 1000


class GameEngine:
    def __init__(self):
        self.lives = 3
        self.score = 0
        self.game_over = False
        self.invulnerable_until = 0
        self.goal_message_until = 0
        self.attempt_started_at = pygame.time.get_ticks()
```

```
    self._build_entities()

def _build_entities(self):
    start_col = GRID_COLS // 2

    self.frog = Frog(
        col=start_col,
        row=START_ROW,
        start_col=start_col,
        start_row=START_ROW,
        cols=GRID_COLS,
        start_row_limit=START_ROW,
    )

    frog_x_range = (
        start_col * CELL_SIZE,
        start_col * CELL_SIZE + CELL_SIZE,
    )

    self.vehicles = []

    for i, row in enumerate(ROAD_ROWS):
        speed = LANE_SPEEDS[i % len(LANE_SPEEDS)]
        vehicle_width = 40 if i % 2 == 0 else 70
        spacing = 300
        count = 2

        for _attempt in range(20):
```

```
    phase = random.randint(0, spacing - 1)
    positions = []
    safe = True

    for n in range(count):
        offset = phase + n * spacing
        x = (
            offset
            if speed > 0
            else WIDTH - offset - vehicle_width
        )
        positions.append(x)

        if not (
            x + vehicle_width <= frog_x_range[0]
            or x >= frog_x_range[1]
        ):
            safe = False

    if safe:
        break

for x in positions:
    self.vehicles.append(
        Vehicle(
            x=x,
            row=row,
            width=vehicle_width,
```

```
                height=CELL_SIZE - 8,
                speed=speed,
            )
        )

def _reset_attempt_timer(self, current_time=None):
    """Start a fresh 30-second attempt."""
    if current_time is None:
        current_time = pygame.time.get_ticks()

    self.attempt_started_at = current_time

def _finish_attempt(self, current_time):
    """Respawn after a failed attempt and deduct exactly one life."""
    self.lives -= 1
    self.frog.reset()
    self._reset_attempt_timer(current_time)

    self.invulnerable_until = (
        current_time + COLLISION_COOLDOWN_MS
    )

    if self.lives <= 0:
        self.lives = 0
        self.game_over = True

def handle_keydown(self, key):
    if key == pygame.K_r:
```

```
            self.lives = 3
            self.score = 0
            self.game_over = False
            self.invulnerable_until = 0
            self.goal_message_until = 0
            self._build_entities()
            self._reset_attempt_timer()
            return

        if self.game_over:
            return

        if key == pygame.K_UP:
            self.frog.move(0, -1)
        elif key == pygame.K_DOWN:
            self.frog.move(0, 1)
        elif key == pygame.K_LEFT:
            self.frog.move(-1, 0)
        elif key == pygame.K_RIGHT:
            self.frog.move(1, 0)

    def update(self):
        if self.game_over:
            return

        current_time = pygame.time.get_ticks()

        for vehicle in self.vehicles:
```

```
            vehicle.update(road_width_px=WIDTH)

        # Handle vehicle collision.
        if (
            current_time >= self.invulnerable_until
            and check_collision(self.frog, self.vehicles)
        ):
            self._finish_attempt(current_time)
            return

        # Handle timeout.
        elapsed = current_time - self.attempt_started_at

        if elapsed >= ATTEMPT_TIME_MS:
            self._finish_attempt(current_time)
            return

        # Handle successful goal.
        if self.frog.row == GOAL_ROW:
            self.score += GOAL_POINTS
            self.goal_message_until = (
                current_time + GOAL_MESSAGE_DURATION
            )
            self.frog.reset()
            self._reset_attempt_timer(current_time)

    def draw(self, surface, font):
        from game import renderer
```

```
renderer.draw_scene(surface, self.frog, self.vehicles)

current_time = pygame.time.get_ticks()
elapsed = current_time - self.attempt_started_at
remaining_ms = max(0, ATTEMPT_TIME_MS - elapsed)

seconds_remaining = (
    remaining_ms + 999
) // 1000

renderer.draw_text(
    surface, font, f"Lives: {self.lives}", (10, 10)
)
renderer.draw_text(
    surface, font, f"Score: {self.score}", (150, 10)
)
renderer.draw_text(
    surface, font,
    f"Time: {seconds_remaining}s",
    (270, 10),
)
renderer.draw_text(
    surface, font,
    "Arrow keys to move. R to restart.",
    (10, HEIGHT - 24),
)
```

```
        if self.game_over:
            renderer.draw_banner(
                surface, font, "GAME OVER - Press R to restart"
            )
        elif current_time < self.goal_message_until:
            renderer.draw_banner(
                surface, font,
                f"GOAL REACHED! +{GOAL_POINTS} points"
            )
```

6.4 Explanation of the timer implementation

Timer duration

ATTEMPT_TIME_MS = 30_000

The timer duration is 30,000 milliseconds, equivalent to 30 seconds.

Start time

self.attempt_started_at = pygame.time.get_ticks()

This records the start of the attempt.

Calculate elapsed time

elapsed = current_time - self.attempt_started_at

The difference between the current time and the start time gives the elapsed time.

Detect timeout

```
if elapsed >= ATTEMPT_TIME_MS:
    self._finish_attempt(current_time)
    return
```

When the elapsed time reaches 30 seconds, the attempt ends.

Deduct one life

self.lives -= 1

The shared _finish_attempt() method handles one life deduction, frog respawn, timer reset, and Game Over.

Display remaining time

remaining_ms = max(0, ATTEMPT_TIME_MS - elapsed)

seconds_remaining = (

    remaining_ms + 999

) // 1000

This calculates the remaining time and rounds the display upward so the countdown starts at 30 seconds.

Restart the timer

self._reset_attempt_timer(current_time)

The timer is reset after a collision, timeout, or successful goal.

6.5 Task 4 testing checklist

1. Timer starts at 30 seconds.
2. Timer decreases with real elapsed time.
3. Timer reaches zero after 30 seconds.
4. Timeout deducts exactly one life.
5. Frog respawns and timer resets after timeout.
6. Vehicle collision still deducts one life.
7. Successful goal adds 10 points and resets the timer.
8. Zero lives displays Game Over and stops normal gameplay.
9. Pressing R resets lives, score, and timer.

7. FINAL INTEGRATION TESTING

After implementing all four tasks, run the game:

py main.py

Check the following:

Test	**Expected result**
Game startup	3 lives, score 0, timer 30 seconds

Arrow-key movement	Frog moves correctly
Vehicle collision	Collision is detected
Collision response	One life deducted and frog respawns
Successful goal	Score increases by 10
Timer	Counts down in real time
Timeout	One life deducted and attempt resets
Zero lives	Game Over appears
Restart	Lives, score, and timer reset

If any test fails, correct the issue and rerun the affected test as well as the related regression tests.

8. FINAL VIDEO RECORDING

The lab requires a 10-second recording of the original game and another 10-second recording of the completed game.

Before video

Filename:

frogger_before.mp4

It should show the original game before modifications.

After video

Filename:

frogger_after.mp4

It should show the completed game, preferably including the frog movement, vehicles, lives, score, and countdown timer.

The videos should be actual recordings of the running application.

9. GITHUB SUBMISSION

Push the completed code to your personal GitHub repository.

If your repository is already connected and the changes are ready to commit:

git add .

git commit -m "Complete Frogger Vibe Coding lab"

git push

Do not create a pull request to the original SETAPESU26 repository.

Upload the following under the Lab-4 folder of your lab repository:

- frogger_before.mp4
- frogger_after.mp4
- Updated Frogger source code
- This AI chat history document

Include all the source files required to run the game.

10. CONCLUSION

The Frogger Vibe Coding lab involved modifying an existing Python Pygame application using AI-assisted coding.

The four tasks covered:

1. **Collision detection:** Replace grid-position comparison with rectangle-intersection testing.
2. **Lives and respawning:** Add three lives, collision cooldown, respawn behavior, and Game Over.
3. **Goal detection and score tracking:** Award points when the frog reaches the goal row.
4. **Countdown timer:** Implement a 30-second timer based on real elapsed time.

The final implementation must be tested to confirm that the new features work together and that the existing game functionality is preserved.

The lab is complete when the updated code runs correctly, the before and after videos are recorded, the AI chat history is saved, and all required deliverables are uploaded to the appropriate GitHub repository.